

Node.js Local Module

✓ Local modules are modules created locally in your Node.js application. These modules include different functionalities of your application in separate files and folders. You can also package it and distribute it via NPM, so that Node.js community can use it. For example, if you need to connect to MongoDB and fetch data then you can create a module for it, which can be reused in your application.



Writing Simple Module

Let's write simple logging module which logs the information, warning or error to the console.

In Node.js, module should be placed in a separate JavaScript file. So, create a Log.js file and write the following code in it.

→ Log.js → A separate file which stores & export the object as a module.

```
var log = {  
  info: function (info) {  
    console.log('Info: ' + info);  
  },  
  warning: function (warning) {  
    console.log('Warning: ' + warning);  
  },  
  error: function (error) {  
    console.log('Error: ' + error);  
  }  
};  
  
module.exports = log
```

log is an object with 3 functionalities.

→ It exports/exports a log obj as a module

In the above example of logging module, we have created an object with three functions - info(), warning() and error(). At the end, we have assigned this object to **module.exports**. The module.exports in the above example exposes a log object as a module.

The **module.exports** is a special object which is included in every JS file in the Node.js application by default. Use **module.exports** or **exports** to expose a function, object or variable as a module in Node.js.

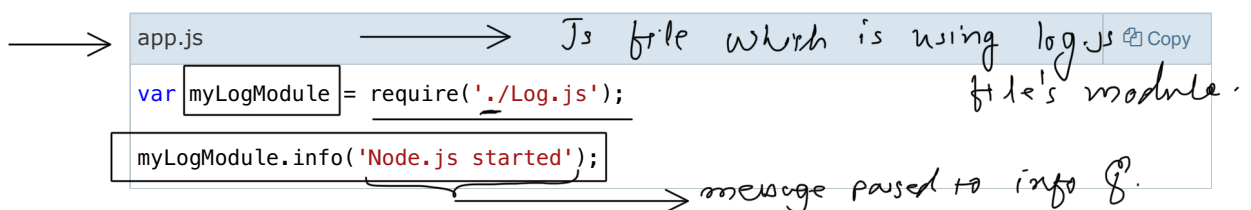
Now, let's see how to use the above logging module in our application.



Loading Local Module

To use local modules in your application, you need to load it using require() function in the same way as core module. However, you need to specify the path of JavaScript file of the module.

The following example demonstrates how to use the above logging module contained in Log.js.



In the above example, app.js is using log module. First, it loads the logging module using require() function and specified path where logging module is stored. Logging module is contained in Log.js file in the root folder. So, we have specified the path './Log.js' in the require() function. The '.' denotes a root folder.

The require() function returns a log object because logging module exposes an object in Log.js using module.exports. So now you can use logging module as an object and call any of its function using dot notation e.g myLogModule.info() or myLogModule.warning() or myLogModule.error()

Run the above example using command prompt (in Windows) as shown below.

```
C:\> node app.js  
Info: Node.js started
```

Thus, you can create a local module using module.exports and use it in your application.

Let's see how to expose different types as a node module using module.exports in the next section.

Want to check how much you know Node.js?

Start Node.js Test